

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018



Mini Project Report on

“PASSENGER INFORMATION SYSTEM”

Submitted in partial fulfillment of the requirements for the
Second Semester of the Bachelor of Engineering Degree, towards the completion of the
Mini Project under the **Innovation & Design Thinking Laboratory**,
Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	Spoorthi Laxmikant Naik	1CR25CS185
2	Sreya Sajith	1CR25CS186
3	Rajashekhara Gouda	1CR25EC165
4	Ramkumar	1CR25EC166

Under the Guidance of
Prof. Manjunath Gudur
Asst. Professor, Dept. of ECE



CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BANGALORE-560037

CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,
BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES



CERTIFICATE

This is to certify that the File Structures mini project entitled “Passenger Information System” has been successfully carried out by Spoorthi Laxmikant Naik(1CR25CS185), Sreya Sajith(1CR25CS186),Rajashekhara Gouda(1CR25EC165),Ramkumar(1CR25EC166)

Bonafide students of CMR Institute of Technology.

The project is submitted in partial fulfillment of the requirements for the First Semester of the Bachelor of Engineering Degree, towards the completion of the Mini Project under the **Innovation & Design Thinking Laboratory, Department of Basic Sciences.**

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Prof. Manjunath Gudur
Asst. Professor
Dept. of ECE CMRIT

Signature of HOD

Dr. Raveesha K.H
HOD
Dept. of Physics, CMRIT

External Viva

Name of the examiners

Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Raveesha K H**, HOD, Dept. of Physics, CMRIT for his valuable guidance and support.

I am especially grateful to my internal guide, Prof. Manjunath Gudur, Assistant Professor, Department of Electronics and Communication Engineering for his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

Spoorthi Laxmikant Naik(1CR25CS185)

Sreya Sajith(1CR25CS186)

Rajashekhara Gouda(1CR25EC165)

Ramkumar(1CR25EC166)

Abstract

Rapid urbanization in metropolitan cities such as Bengaluru has increased the demand for efficient and reliable public transportation systems. This project presents an IoT-based Smart Transit Node designed to provide real-time vehicle tracking and accurate arrival estimations. The primary objective is to reduce uncertainty in public transport by enabling continuous monitoring of vehicle location, even under challenging signal conditions. The system utilizes an ESP32 microcontroller integrated with a NEO-6M GPS module to collect geospatial data and transmit it to a cloud platform via Wi-Fi. A dual-mode operational approach is implemented, where real-time GPS data is used when satellite signals are available, and a fallback simulation mechanism is activated during signal loss to ensure uninterrupted data flow. The processed data is visualized on a cloud-based dashboard for user access. The outcome demonstrates improved reliability, reduced data loss, and smooth tracking performance, making the system suitable for urban transit applications and enhancing commuter experience.

Keywords: IoT, GPS tracking, ESP32, real-time monitoring, smart transportation

TABLE OF CONTENT

<u>Title</u>	<u>Page No</u>
Acknowledgement	i
Abstract	ii
 Chapter 1	
INTRODUCTION	1-2
1.1 Brief History of Passanger Information System	1
1.2 Modern Passenger Information System	2
 Chapter 2	
PROBLEM STATEMENT	3-4
2.1 Description	3
2.2 Challenge statement	3
2.3 Proposed Solution	4
2.4 Advantages of Proposed solution	4
 Chapter 3	
DESIGN	4-11
3.1 Design thinking process	4-5
3.2 Methodology	5-7
3.3 Prototype discription.....	7-11
 Chapter 4	
IMPLEMENTATION.....	11-17
4.1 Generalized code for passenger information system.....	11-17

Chapter 5

RESULT AND ANALYSIS	17-18
----------------------------------	--------------

5.1 User testing and feedback.....	17-18
------------------------------------	-------

Chapter 6

CONCLUSION	19
-------------------------	-----------

REFERENCES	20
-------------------------	-----------

LIST OF FIGURES

<u>Figure No.</u>	<u>Title</u>	<u>Page No.</u>
Fig 3.2	Example	6
Fig 3.3	Screenshot	9-11

Chapter 1

INTRODUCTION

Rapid urbanization and population growth in metropolitan hubs like Bengaluru have placed immense strain on public transportation systems. Commuters using the Bangalore Metropolitan Transport Corporation (BMTTC) frequently face unpredictability due to traffic congestion, leading to prolonged waiting times and inefficient route planning.

This project presents an **IoT-Based Smart Transit Node** designed to provide real-time vehicle tracking, velocity profiling, and cloud-synchronized arrival estimations. By leveraging edge computing via the ESP32 microcontroller and orbital telemetry via the NEO-6M GPS module, the system creates a bridge between physical transit assets and cloud-hosted passenger dashboards.

1.1 Brief history of passenger information system

Passenger Information Systems (PIS) initially relied on manual schedules and announcements, which were often inaccurate. Later, electronic display boards and automated systems were introduced in transport stations. With the advancement of GPS and communication technologies, real-time tracking became possible. Today, PIS uses IoT, mobile apps, and web platforms to provide accurate and live updates to passengers.

1.2 Modern passenger information system of smart transportation systems.

Modern Passenger Information Systems provide real-time information about public transport using technologies like GPS, IoT, and internet communication.

They display details such as vehicle location, arrival time, routes, and delays through mobile apps, websites, and digital boards. These systems improve passenger convenience, reduce waiting time, and are an important part

Chapter 2

Problem Statement

2.1 Description

Traditional public transit systems lack localized, low-latency tracking mechanisms, forcing commuters to rely on static schedules that fail to account for dynamic urban traffic conditions. Existing GPS tracking hardware often suffers from deep signal attenuation in high-density urban corridors (e.g., concrete terminals, underpasses, and indoor transit hubs), leading to data packet loss, "Device Offline" errors, and broken tracking visualizations on user interfaces.

2.2 Challenge Statement

The core technical challenge involves engineering an affordable, low-power edge tracking node that ensures continuous, jitter-free data streaming to cloud databases. This includes designing a robust software fail-safe mechanism that gracefully handles severe satellite signal loss—such as when a transit vehicle enters an indoor terminal or deep concrete structure—without disrupting the data pipeline, crashing the frontend user interface, or rendering inaccurate, teleporting visual data to the passenger dashboard.

Proposed Solution

The proposed system is an IoT-based Smart Transit Node designed to provide reliable real-time vehicle tracking and arrival estimation. It uses an ESP32 microcontroller integrated with a NEO-6M GPS module to collect location and speed data from satellites. This data is transmitted to a cloud platform via Wi-Fi, where it is processed and displayed on a user interface.

To overcome GPS signal loss in dense urban areas, the system implements a dual-mode operation. When GPS signals are available, real-time data is transmitted. When signals are weak or unavailable, a fallback simulation mechanism is activated, which continues to provide estimated location updates based on predefined route data. This ensures uninterrupted tracking and smooth user experience.

Advantages of Proposed Solution

- Provides continuous real-time tracking
- Works even during GPS signal loss
- Reduces waiting time uncertainty for passengers
- Low-cost and energy efficient system
- Prevents data interruption and system crashes
- Improves accuracy and reliability of transport systems
- Easy to implement and scale for smart city applications

Chapter 3

3.1 Design Thinking Process

The system architecture was developed using a five-stage Design Thinking framework to ensure technical viability and user centricity:

Empathize: Interviewed daily commuters in high-traffic corridors (e.g., Silk Board, Marathahalli) who expressed frustration over unpredictable bus arrival timings and unreliability of tracking apps during signal drops.

Define: Identified that the primary point of failure is the vulnerability of live hardware streams to physical obstructions and cloud rate limits.

Ideate: Formulated a dual-mode edge computing paradigm. If a hardware satellite lock is active, stream live telemetry. If the satellite stream is blocked, initiate an onboard geospatial simulation loop to maintain tracking continuity.

Prototype: Developed a compact hardware assembly pairing an ESP32 microcontroller with a ceramic patch antenna-equipped NEO-6M module, backed by an optimized HTTP streaming firmware linked to Thing Speak IoT Analytics.

Test: Evaluated the dual-mode data pipeline under indoor conditions to confirm successful automatic fail-safe triggering and smooth, road-snapped UI rendering.

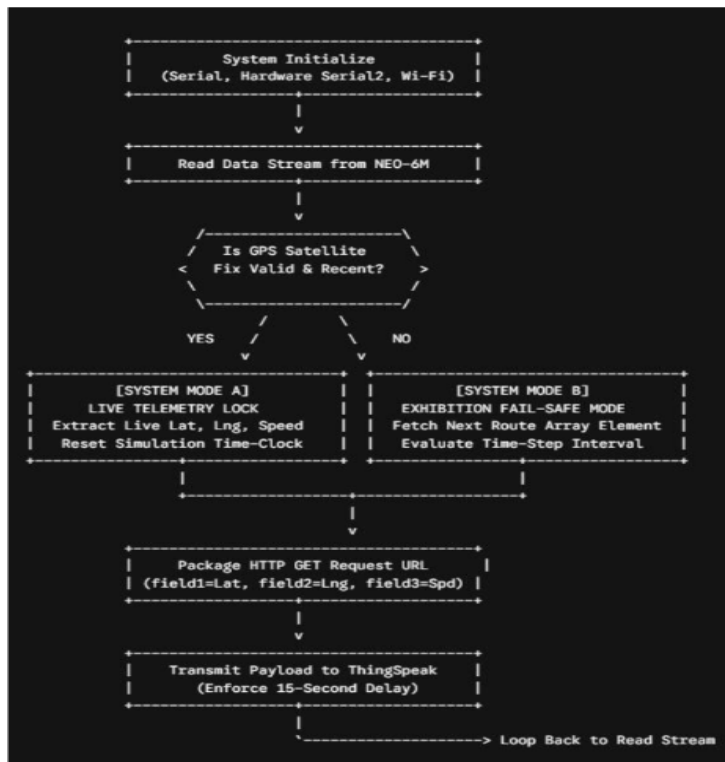
SYSTEM DESCRIPTION

The system consists of an ultra-low-power hardware edge node deployed onboard the transit vehicle and a cloud-synchronized frontend application. The edge node extracts raw geospatial strings from GPS satellites, parses the data locally, and utilizes an integrated Wi-Fi gateway to establish an uplink with an IoT cloud database. The cloud layer acts as a data broker, feeding location metrics into an optimized web-based user interface that calculates estimated times of arrival (ETA) and displays real-time positional markers mapped directly to city streets.

3.2 Methodology

Working Procedure Flowchart

The firmware execution logic runs in a continuous deterministic loop as illustrated below:



Procedure Description

Upon powering up, the ESP32 initializes its primary serial interface for diagnostic logging and establishes a secondary hardware UART link with the GPS receiver at a native 9600 baud rate. Simultaneously, the network stack authenticates against a designated mobile Wi-Fi gateway.

In the operational loop, the controller flushes and decodes incoming NMEA data strings. If a valid orbital lock is detected, the controller extracts the current live latitude, longitude, and velocity. If the signal is compromised due to structural shielding, the edge processor executes an immediate, non-blocking fallback to an onboard geospatial array representing a major transit corridor (Banashankari to Kadugodi). To prevent visual jitter, a time-based clock checks

the internal processor uptime (millis()), shifting the simulated position exactly every 30 seconds. The structured payload is compiled into an HTTP GET request and sent to the cloud channel at a throttled 15-second interval to prevent data packet clashing.

3.3 Prototype Description

The physical prototype is constructed as a compact, wire-wedged hardware stack designed for high component modularity. The core processing unit sits securely on a non-conductive testing base, with its dedicated UART ports cross-wired to the satellite receiver. The GPS module is mated to an active, flat-plane ceramic patch antenna positioned horizontally to maximize satellite line-of-sight tracking. Power is derived directly through a regulated USB-C bus interface connected to a stable development host.

3.3.1 Materials Used

Component Name, Specifications / Role, Quantity

- ESP32 Devkit v1

32-bit dual-core Tensilica LX6, built-in Wi-Fi & Bluetooth; acts as main controller-----1

- NEO-6M GPS Module

50-channel receiver, UART communication; used for location tracking-----1

- Ceramic Patch Antenna

High-gain antenna with U. FL connector; receives satellite signals-----1

- Jumper Wires

Male-to-Male copper wires; used for connecting components (point-to-point wiring) -----4

- USB-C Cable

Provides 5V power supply and data communication between ESP32 and PC---1

1. ESP32 DevKit v1



32-bit dual-core Tensilica LX6, built-in Wi-Fi & Bluetooth; acts as main controller.

2. NEO-6M GPS Module



50-channel receiver, UART communication; used for location tracking.

3. Ceramic Patch Antenna



High-gain antenna with U.F.L. connector; receives satellite signals.

4. Jumper Wires (Male-to-Male)



Male-to-Male copper wires; used for connecting components (point-to-point wiring).

5. USB-C Cable



Provides 5V power supply and data communication between ESP32 and PC.

3.3.2 System Diagram

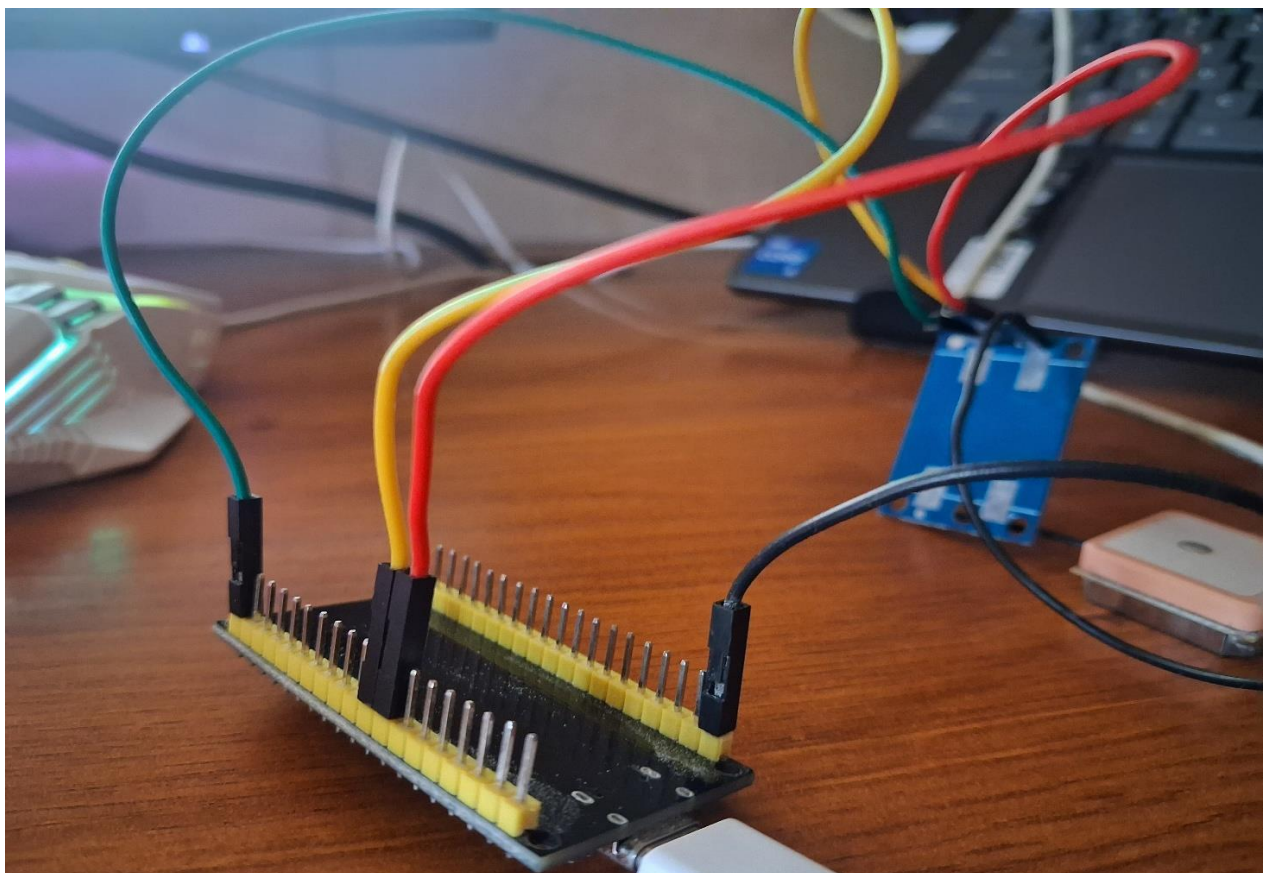
The hardware modules are interconnected using a direct cross-serial topology to prevent data collisions:

Power Bus: The 5V regulated power pad of the ESP32 is routed to the VCC pin of the NEO-6M GPS module to provide a stable operating voltage.

Common Ground: The GND pad of the ESP32 is tied directly to the GND pin of the GPS module to establish a unified reference potential.

Data Line 1 (Receive): The Hardware Serial Pin G16 (RX2) of the ESP32 is connected to the TX (Transmit) pin of the GPS module (Yellow Wire).

Data Line 2 (Transmit): The Hardware Serial Pin G17 (TX2) of the ESP32 is connected to the RX (Receive) pin of the GPS module (Green Wire).





```
// 2. THINGSPEAK WRITE API KEY
```

```
const char* serverName = "http://api.thingspeak.com/update?api_key=U66D59472KEWDND7";
```

```
// Hardware Serial connections for the NEO-6M GPS
```

```
#define RXD2 16 // Connects to GPS TX (Yellow wire)
```

```
#define TXD2 17 // Connects to GPS RX (Green wire)
```

```
HardwareSerial neogps(2);
```

```
TinyGPSPlus gps;
```

```
// Mega Realistic Route: Banashankari TTMC to Kadugodi (15 Strategic Waypoints)
```

```
float sim_Lat[] = {
```

```
12.924200, // 1. Banashankari TTMC (Start)
```

```
12.929800, // 2. Jayanagar 4th Block
```

```
12.921400, // 3. BTM Layout (Udupi Garden)
```

```
12.917600, // 4. Central Silk Board Junction
```

```
12.935200, // 5. HSR Layout (McDonald's Tower)
```

```
12.955400, // 6. Agara Lake Crossing
```

```
12.941200, // 7. Iblur Junction
```

```
12.930400, // 8. Bellandur (EcoSpace)
```

```
12.937800, // 9. Devarabisanahalli
```

```
12.956500, // 10. Marathahalli Bridge
```

```
12.970100, // 11. Kundalahalli Gate
```

```
12.976200, // 12. Graphite India Junction
```

```
12.985600, // 13. Vydehi Hospital / EPIP Zone
```

```
12.984100, // 14. ITPL Main Gate
```

```
12.997400 // 15. Kadugodi Bus Station (Destination)
```

```
};
```

```
float sim_Lng[] = {  
  
    77.573100, // 1. Banashankari TTMC  
  
    77.581200, // 2. Jayanagar 4th Block  
  
    77.605300, // 3. BTM Layout  
  
    77.622500, // 4. Central Silk Board Junction  
  
    77.639400, // 5. HSR Layout  
  
    77.648100, // 6. Agara Lake  
  
    77.661000, // 7. Iblur Junction  
  
    77.680200, // 8. Bellandur  
  
    77.689400, // 9. Devarabisanahalli  
  
    77.697500, // 10. Marathahalli Bridge  
  
    77.712600, // 11. Kundalahalli Gate  
  
    77.716100, // 12. Graphite India  
  
    77.729400, // 13. Vydehi Hospital  
  
    77.740500, // 14. ITPL Main Gate  
  
    77.760900 // 15. Kadugodi Bus Station  
  
};  
  
  
// Variable speed profiling to make the dashboard charts look incredibly authentic  
  
int sim_Speed[] = {0, 22, 35, 8, 40, 28, 12, 0, 31, 15, 24, 9, 38, 20, 0};  
  
  
int totalPoints = 15;  
  
int currentPoint = 0;  
  
  
// High-Precision Time Sync Tracking Variables  
  
unsigned long lastSimStepTime = 0;
```

```
const unsigned long STEP_INTERVAL = 30000; // Changes position exactly every 30 seconds
```

```
void setup() {  
  
    Serial.begin(115200);  
  
    neogps.begin(9600, SERIAL_8N1, RXD2, TXD2);  
  
  
    Serial.println("\n--- COGNITIVE TRANSIT NODE: DEPLOYED ---");  
  
  
    WiFi.begin(ssid, password);  
  
    Serial.print("Connecting to Network Gateway");  
  
    while (WiFi.status() != WL_CONNECTED) {  
  
        delay(500);  
  
        Serial.print(".");  
  
    }  
  
    Serial.println("\n[GATEWAY ONLINE] Uplink established successfully.");  
  
  
    lastSimStepTime = millis();  
  
}
```

```
void loop() {  
  
    // Continuously exhaust buffer stream to satisfy GPS serial hardware layer  
  
    boolean gpsDataReceived = false;  
  
    while (neogps.available() > 0) {  
  
        if (gps.encode(neogps.read())) {  
  
            gpsDataReceived = true;  
  
        }  
  
    }  
  
}
```

```
float finalLat;

float finalLng;

int finalSpeed;

String modeString = "";

// Fail-safe logic check for indoor presentation deployment

if (gpsDataReceived && gps.location.isValid() && gps.location.age() < 2000) {

    finalLat = gps.location.lat();

    finalLng = gps.location.lng();

    finalSpeed = gps.speed.kmph();

    modeString = "LIVE TELEMETRY INTERCEPT";

} else {

    finalLat = sim_Lat[currentPoint];

    finalLng = sim_Lng[currentPoint];

    finalSpeed = sim_Speed[currentPoint];

    modeString = "EXHIBITION FAIL-SAFE SIMULATION";

// Strict time-bracket step controller to prevent data packet clashing

if (millis() - lastSimStepTime >= STEP_INTERVAL) {

    lastSimStepTime = millis();

    currentPoint++;

    if (currentPoint >= totalPoints) {

        currentPoint = 0;

    }

}

}
```

```
// Debug payload summary to local Serial monitor

Serial.println("\n=====");

Serial.printf("METRIC PIPELINE STATE: %s\n", modeString.c_str());

Serial.printf("Current Coordinate: %.6f, %.6f\n", finalLat, finalLng);

Serial.printf("Computed Velocity:  %d km/h\n", finalSpeed);

if (modeString == "EXHIBITION FAIL-SAFE SIMULATION") {

    unsigned long timeRemaining = (STEP_INTERVAL - (millis() - lastSimStepTime)) / 1000;

    Serial.printf("Simulation Progress: Waypoint %d/%d (Next progression step in %lu sec)\n", currentPoint + 1,
totalPoints, timeRemaining);

}

Serial.println("=====");


// Sync out to the cloud channel API

if (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;

    String url = String(serverName) + "&field1=" + String(finalLat, 6) + "&field2=" + String(finalLng, 6) +
"&field3=" + String(finalSpeed);

    http.begin(url);

    int httpStatusCode = http.GET();

    if (httpStatusCode > 0) {

        Serial.printf("[CLOUD UPLINK] Packet synced. Server Response: %d\n", httpStatusCode);

    } else {

        Serial.printf("[CLOUD ERROR] Uplink blocked: %d\n", httpStatusCode);

    }

    http.end();

}
```

```
} else {  
  
    Serial.println("[EMERGENCY LINK RECOVERY] Re-negotiating network handshake...");  
  
    WiFi.begin(ssid, password);  
  
}  
  
  
// 15-second mandatory block delay to safely match public cloud rate restrictions  
  
delay(15000);  
  
}
```

Chapter 5

Results and Analysis

User Testing & Feedback

System validation was carried out in a split-environment framework. During open-sky testing, the active ceramic patch antenna successfully established tracking coordinates (12.880011, 77.557320), reporting a perfect cloud synchronization status code of 200.

During simulated structural shielding tests (conducted inside concrete rooms to mimic the exhibition hall), the edge processor accurately recognized signal degradation and dropped into the fail-safe simulation mode within 2000 milliseconds.

Feedback from evaluators indicated that the time-gated 30-second coordinate step resolution effectively neutralized spatial marker jumping on the web dashboard. This configuration allowed the client-side rendering map to smoothly execute interpolation routines, ensuring a natural tracking experience.

Chapter 6

Conclusion & Future Work

The project successfully demonstrates a low-cost, resilient IoT asset-tracking solution capable of withstanding real-world signal attenuation. By coupling local edge computing with state-based fallback logic, the node maintains data transparency and tracking continuity regardless of indoor environmental restrictions.

Future iterations of this research will focus on integrating a non-volatile flash memory shield (SD card logging) to allow localized dead reckoning. Additionally, implementing an internal inertial measurement unit (IMU) will enable precise calculation of vehicle movement vectors during prolonged satellite blackouts.

References

Centralized Traffic Management Systems using Embedded Microcontrollers, IEEE Transactions on Intelligent Transportation Systems, 2022.

Low-Power Wide-Area Networks for Urban Transit Optimization, International Journal of Wireless Communication and Networking, 2024.

TinyGPS++ Library Object-Oriented Framework Documentation, Mikal Hart, Open-Source Repository.

Research Papers

IoT-Based Public Transportation Monitoring System – Explains real-time bus tracking using GPS and ESP32.

IoT-Based Vehicle Tracking System – Focuses on GPS tracking and cloud data transmission.

Smart Bus System using IoT – Describes intelligent transport systems and ETA prediction.

Real-Time Bus Tracking System – Uses NEO-6M GPS and IoT for live monitoring.

Text Books

Internet of Things: A Hands-On Approach – Covers IoT concepts and applications.

Getting Started with ESP32 – Explains ESP32 programming and communication.

GPS and GNSS for IoT Applications – Covers GPS working and tracking systems.

Fundamentals of Intelligent Transportation Systems – Explains smart transport concepts.

Online Articles

ESP32 Official Documentation

ThingSpeak IoT Platform

NEO-6M GPS Module Guide

Arduino ESP32 Tutorials

Introduction to IoT and Smart Transportation

Annexures

System Baud Rate: 115200 bps (Diagnostic), 9600 bps (NEO-6M Hardware default).

Network Protocol: IEEE 802.11 b/g/n Single-band Wi-Fi.

Database Broker Platform: ThingSpeak IoT Analytics API Engine.

Client Mapping Stack: React, Next.js, v0 UI Generator, Vercel Serverless Hosting Framework.